Internship Report On

“NTPWEdit Clone - Windows Password Management Tool”

In the partial fulfillment of the requirement for Third Year Computer Engineering

Submitted by Kakad Saurabh Madhukar Roll No: 32 Rohan Sachin Pagar Roll No: 32

Under the Guidance of Prof. P. P. Kakade

DEPARTMENT OF COMPUTER ENGINEERING

NGSPM’S BRAHMA VALLEY COLLEGE OF ENGINEERING AND RESEARCH INSTITUTE NASHIK-422213

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE 2025-2026

ACKNOWLEDGEMENT

It is a great pleasure for us to express our deep sense of gratitude to all those who have helped us directly and indirectly in the successful completion of this internship project. Technical projects of this nature require not only theoretical knowledge but also a significant amount of hands-on experimentation and research into undocumented systems.

We would like to express our special thanks of gratitude to our guide, **Prof. P. P. Kakade**, for their valuable guidance and constant encouragement during the development of this project. Their insights into systems programming and technical expertise were instrumental in overcoming the various challenges we faced, especially regarding the binary parsing of system files.

We also express our sincere thanks to the **H.O.D. of Computer Engineering Department** and the **Principal of Brahma Valley College of Engineering and Research Institute, Nashik**, for providing us with the necessary facilities and a conducive environment for our project work. The resources provided by the institution allowed us to dive deep into the project requirements.

Finally, we would like to thank our parents and friends for their continuous support and motivation throughout the internship duration. Their encouragement was a great source of strength during the debugging phases of this complex utility.

Kakad Saurabh Madhukar Rohan Sachin Pagar

ABSTRACT

The Security Accounts Manager (SAM) hive is a critical component of the Windows NT operating system family, storing sensitive user account information, including password hashes and group memberships. Management of these accounts typically requires the operating system to be active; however, in scenarios involving system lockouts, forgotten administrative credentials, or forensic investigations, offline access is required. These scenarios often occur in disaster recovery situations where the OS cannot be booted into a functional state.

This internship project, titled **"NTPWEdit Clone"**, involves the development of a lightweight, standalone C++ application designed to parse and manipulate Windows Registry hives directly at the binary level. By bypassing the Windows Registry APIs (which are restricted by kernel-level locks on active systems), the tool can operate on SAM files located on non-booted drives. This is achieved through a custom-built registry parsing engine that treats the hive as a raw binary database.

The project demonstrates a modular approach to system-level programming, featuring a custom registry hive parsing engine (`Hive` class), a SAM-specific data manager (`SAMManager`), and a native Win32 user interface. The report details the technical implementation of registry record structures (nk, vk, sk, etc.), the logic behind user account enumeration, and the challenges of building a dependency-free system tool using the MinGW toolkit. The resulting utility automates SAM detection and provides a clear, high-performance interface for account visualization, serving as a powerful tool for system administrators and security professionals.

INDEX

Sr. No.	Content	Page No.
1	**INTRODUCTION**	**1**
1.1	Project Overview and Context	1
1.2	Detailed Problem Statement	3
1.3	Objectives of the Project	5
1.4	Scope and Technical Boundaries	7
1.5	Development Methodology	9
2	**LITERATURE SURVEY / BACKGROUND**	**12**
2.1	Evolution of Configuration Systems in Windows	12
2.2	Windows Security Architecture: An In-Depth View	15
2.3	The Role of the LSASS and SAM Modules	18
2.4	Cryptographic Evolution: From LM to NTLMv2	21
2.5	Theoretical History of Registry Research	24
3	**REGISTRY HIVE FORMAT DEEP DIVE**	**28**
3.1	The Binary Philosophy of Registry Bins	28
3.2	Detailed Analysis of the REGF Header	31
3.3	Cell Management and Memory Segmentation	34
3.4	Structural Logic of Named Key (nk) Records	37
3.5	Value Key (vk) and Data Indirection	40
3.6	Search Optimization: Hash Lists (lh/lf)	43
4	**SYSTEM FLOW AND ARCHITECTURE**	**47**
4.1	High-Level Operational Sequence	47
4.2	Component-Based Architecture and Layering	50
4.3	Resiliency and Error Handling Philosophy	53

4.4	Functional Flow Diagrams	56
5	**TECHNICAL IMPLEMENTATION - REGISTRY ENGINE**	**60**
5.1	Registry Engine (ntreg.cpp) Logic Walkthrough	60
5.2	Cell Level Access and Binary Pointer Arithmetic	63
5.3	Enumerate Subkeys Algorithm Analysis	66
5.4	Value Key Retrieval and Data Type Mapping	69
6	**TECHNICAL IMPLEMENTATION - SAM MANAGER**	**73**
6.1	SAM Management (sam.cpp) Responsibilities	73
6.2	Recursive Path Resolution in Hierarchical Hives	76
6.3	User Account Enumeration and RID Mapping	79
6.4	Forensic Analysis of V and F records	82
7	**WIN32 UI ARCHITECTURE**	**86**
7.1	Dialog-Based Design Philosophy and UX Goals	86
7.2	The Win32 Message Pump and Native Controls	89
7.3	ListView Data Binding and Dynamic Updates	92
7.4	Automatic Path Detection Logic for System Config	95
8	**BUILD SYSTEM AND AUTOMATION**	**99**
8.1	MinGW Toolkit Implementation and Toolchain	99
8.2	Static Linking for Dependency-Free Portability	102
8.3	Resource Compilation with windres and RC Files	105
9	**TROUBLESHOOTING AND FAQ**	**109**
9.1	Common Error Codes and Resolutions	109
9.2	Permission and Lock Exceptions Management	112
10	**RESULTS AND PERFORMANCE**	**116**
10.1	Functional Verification and Test Case Summary	116
10.2	Performance Benchmarks and Load Analysis	119
11	**DISCUSSION AND SECURITY**	**123**
11.1	Ethical Considerations in Security Software	123
11.2	The Impact of Full Disk Encryption (BitLocker)	126
11.3	Technical Challenges and Future Resilience	129

12	CONCLUSION	133
13	FUTURE SCOPE AND ENHANCEMENTS	136
14	GLOSSARY OF TERMS	140
15	REFERENCES	145

CHAPTER 1: INTRODUCTION

1.1 Project Overview and Context

The Windows Registry is a hierarchical database that stores configuration settings and options on Microsoft Windows operating systems. It contains settings for the hardware, operating system software, most non-operating-system software, users, preferences of the PCs, and system-wide policies. The registry provides a vital window into the operation of the runtime system and acts as the "nervous system" of a Windows machine.

One of the most critical parts of the registry is the **Security Accounts Manager (SAM)**. The SAM hive stores local user accounts, passwords (stored as encrypted hashes), and group membership information. Under normal operating conditions, when the Windows kernel is active, the SAM hive is loaded and exclusively locked by the `System` process. This kernel-level lock prevents any standard user-mode application, or even administrative tools like `Regedit`, from reading or modifying the file directly. This is a deliberate security feature designed to prevent malicious software from stealing or tampering with user credentials while the machine is running.

The **"NTPWEdit Clone"** project is an ambitious endeavor to create a standalone, high-performance C++ application that can read and potentially modify these locked security files by accessing them "offline." This means the tool is designed to work when the operating system is not currently active, such as when the system is booted from a recovery CD, a Linux Live USB, or when the target hard drive is connected to another computer as a secondary drive. By re-implementing the core parsing logic of the registry format from scratch, the NTPWEdit Clone allows for account recovery and management without the need for the original OS's authentication mechanisms.

1.2 Detailed Problem Statement

In the modern computing environment, access control and security are paramount. However, several scenarios arise where these very security mechanisms become a critical hindrance to legitimate system administration:

1. **Forgotten Administrative Credentials**: It is common for users or system administrators to lose or forget local account passwords. In the absence of a domain controller or a cloud-sync backup (like Microsoft Account), this often leads to a "deadlocked" system where the only traditional solution is a complete reinstallation of the operating system, resulting in significant data loss and downtime.
2. **Administrative Lockouts**: Due to improperly configured security policies, malware infections, or brute-force protections, critical administrative accounts (like the built-in 'Administrator') can become disabled or locked out. Re-enabling these accounts through the OS becomes impossible if they are the only accounts with sufficient privileges.
3. **Forensic Investigation and Incident Response**: Security professionals and digital forensic investigators often need to audit account activity on a system that has been compromised. To maintain the integrity of the evidence, they must work on "dead" disk images. They need a tool that can extract user lists and account status without ever booting the compromised OS.
4. **Registry Corruption and Recovery**: Standard OS tools (like `Regedit`) rely on the Windows API, which is notoriously unforgiving of minor binary inconsistencies in the registry hives. A low-level, custom-built parser

can often navigate a partially corrupted hive where the OS would simply crash or fail to boot.

1.3 Objectives of the Project

The primary objectives of the NTPWEdit Clone project are:

1. **Develop a Robust, Custom Registry Parser**: To architect a C++ engine from the ground up that can navigate the proprietary binary format of Windows hives. This involves re-implementing the logic for parsing `regf` headers, `hbin` blocks, and the complex cell-based architecture of `nk` (Named Key) and `vk` (Value Key) records.
2. **Bypass Kernel Locks via Direct Disk I/O**: To implement a file-based access model that treats hived files as raw data, allowing for reading and modification tasks on non-active system hives where the Windows kernel is not exerting its exclusive lock.
3. **Automate System Identification**: To implement intelligent logic that can automatically locate the local system's SAM file by auditing environment variables like `SYSTEMROOT` and querying system APIs to determine the architecture (32-bit vs 64-bit) for path construction.
4. **Accurate User Account Enumeration**: To successfully extract the list of usernames and their corresponding Relative Identifiers (RIDs), which are essential for identifying system accounts vs. user-created accounts.
5. **Native, Portable User Interface**: To provide a responsive, clean interface using the native Win32 API, ensuring the tool is lightweight and can run even in minimal environments like Windows PE or recovery consoles.
6. **Dependency-Free Compilation**: To use a toolchain (MinGW) that allows for static linking of all necessary runtimes, resulting in a single, portable executable that does not require external DLLs or frameworks like .NET or Java.

1.4 Scope and Technical Boundaries

The scope of this internship project covers the full lifecycle of a system utility:

- **The Binary Layer**: Understanding and mapping the little-endian binary structures of the registry.
- **The Logic Layer**: Implementing tree-walking algorithms to navigate the hierarchical keys.
- **The Domain Layer**: Mapping the specific structure of the SAM hive (SAM -> Domains -> Account -> Users -> Names).
- **The Presentation Layer**: Building a Win32 dialog-based UI with standard controls like `ListView` and `Edit`.
- **The Build Layer**: Automating the compilation of resources and C++ source code.

The project does not aim to replace enterprise-level Active Directory tools but focuses on the local SAM database, which is the foundational security layer for every Windows NT-based machine.

1.5 Development Methodology

The project follows a modified **Agile/Iterative** development lifecycle:

1. **Research and Analysis**: Deep diving into the hex layouts of healthy SAM hives. Using forensic tools to verify the expected behavior of registry records.
2. **Engine Prototyping**: Building the `Hive` class as a standalone binary reader. This was the most critical phase, as every other module depends on the engine's ability to find and read cells.
3. **Domain Logic Integration**: Building the `SAMManager` on top of the `Hive` engine. This involved "walking the tree" to reach the user names.
4. **GUI Integration**: Moving from a console-based output to a proper Win32 dialog.

5. **Testing and Refinement**: Verifying the tool on different Windows versions (XP, 7, 10, 11) to ensure the pathing logic remains robust across OS generations.

CHAPTER 2: BACKGROUND AND LITERATURE SURVEY

2.1 Evolution of Configuration Systems in Windows

In the early days of personal computing, software configuration was a chaotic affair. In the Windows 3.x era, settings were stored in `.ini` files (initialization files). These were simple text files following a specific `[Section]` `Key=Value` format. However, as applications became more complex, "INI Hell" ensued. Different applications would overwrite each other's settings, and the system became slow because the OS had to parse thousands of lines of text every time a setting was needed.

The introduction of the Windows Registry in Windows NT was a paradigm shift. It moved configuration data into a centralized, binary-indexed database. This allowed for:

- **Fast Access**: Binary indexing is orders of magnitude faster than text parsing.
- **Transactional Integrity**: The OS could ensure that settings were either fully written or not written at all, preventing half-written "corrupt" settings.
- **Security**: Unlike text files, binary hives could have sophisticated Access Control Lists (ACLs) applied at a granular level.

2.2 Windows Security Architecture: An In-Depth View

The Windows security model is built around the concept of a **Security Principal** (a user, a group, or a computer). Every action in the system is performed under the context of an **Access Token**.

Key components include:

- **LSASS (Local Security Authority Subsystem Service)**: The core service responsible for enforcing security policies. It verify users logging on to a Windows computer or server, handles password changes, and creates access tokens.
- **SAM (Security Accounts Manager)**: A database used by LSASS to authenticate local users. It is essentially a "local Active Directory."
- **SRM (Security Reference Monitor)**: The kernel-mode component that performs actual access checks on objects (files, keys, threads).

2.3 The Role of the LSASS and SAM Modules

When you type your password on the login screen, Windows doesn't "know" your password. It takes the text you typed, runs it through a one-way hashing function (like MD4), and then LSASS asks the SAM module: "Does this hash match the hash you have on disk for this user?"

If the SAM module returns a "Yes," LSASS generates an Access Token for you. If "No," you are denied entry. The SAM hive is the ultimate source of truth for local security. Because it's so critical, Windows prevents any process other than LSASS from accessing it while the system is running.

2.4 Cryptographic Evolution: From LM to NTLMv2

Computer security is a cat-and-mouse game.

- **LM (LanMan) Hashes**: Used in early Windows versions. These were very weak because they were case-insensitive and split into two 7-character chunks, making them easy to crack with brute force.
- **NTLM (NT Lan Manager)**: Introduced with Windows NT. It uses the MD4 algorithm and is much stronger.
- **NTLMv2**: Adds "challenge-response" mechanisms to prevent "re-play" attacks where a hacker steals a hash and tries to use it without knowing the password.
- **SysKey**: To further protect these hashes, Microsoft introduced SysKey. This is a 128-bit key that encrypts the entire hash section of the SAM. Our project must navigate the structures that would eventually be passed to the decryption engine.

CHAPTER 3: REGISTRY HIVE FORMAT DEEP DIVE

3.1 The Binary Philosophy of Registry Bins

A registry hive is essentially a custom filesystem residing within a single file. It is designed for random access and durability. The file is divided into a **Header** (4KB) and a series of **Bins**.

A **Bin** is a logical container for **Cells**. Bins are always sized in multiples of 4KB and start with the ASCII signature `hbin`. This alignment is crucial for memory-mapped I/O, as it aligns with the standard page size of modern processors.

3.2 Detailed Analysis of the REGF Header

The `regf` header is the identification card of the hive. It resides at the very beginning of the file.

Critical Fields in the Header:

1. **Sequence Numbers**: Windows uses two sequence numbers (`seq1` and `seq2`). When writing to a hive, it increments one, writes the data, and then increments the other. If these two numbers don't match, it means the system crashed in the middle of a write operation, and the hive might be unstable.
2. **Root Key Offset**: This is the "index" to the entire database. It points to the first `nk_record`, which is always the root key.
3. **Checksum**: A security measure to prevent data corruption from being processed.

3.3 Cell Management and Memory Segmentation

Inside a `hbin`, the data is stored in **Cells**. A cell is not just data; it's a wrapper.

- **The Size Prefix**: Every cell starts with a 4-byte signed integer.
- **Positive Size**: Means the cell is free space (a "hole").
- **Negative Size**: Means the cell is in use (allocated). The absolute value of the integer is the actual size.

This clever system allows the Registry to manage its own memory without an external allocation table. If the Registry needs to store a new key, it looks for a "positive size" cell large enough to hold it, splits it, and marks the new cell as "negative."

3.4 Structural Logic of Named Key (nk) Records

The `nk` record is the most important structure for navigation. It contains:

- **Signature**: `nk` (0x6B6E).
- **Flags**: bit 5 set indicates a "root" key.
- **Timestamps**: Last modification time in FILETIME format.

- **Name**: The name of the key (e.g., "Software").
- **Pointers**: Offsets to the subkey list and the value list.

3.5 Value Key (vk) and Data Indirection

A `vk` record stores the metadata of a value.

- **Type**: REG_SZ (string), REG_DWORD (integer), REG_BINARY, etc.
- **Data Offset**: If the data is large (> 4 bytes), this points to another cell containing the actual data.
- **Inline Data**: If the data is 4 bytes or less, the Registry "cheats" and stores the data **inside** the offset field itself. This is an optimization that saves a disk seek.

CHAPTER 4: SYSTEM FLOW AND ARCHITECTURE

4.1 High-Level Operational Sequence

The NTPWEdit Clone operates as a state machine that transitions through various phases of file loading and data interpretation.

1. **Discovery Phase**:
   - The app launches and polls the system environment.
   - It looks for the `%SystemRoot%` variable.
   - It checks for the existence of `C:\Windows\System32\config\SAM`.
2. **Validation Phase**:
   - The user clicks "Open."
   - The `Hive` class opens the file stream.
   - It performs the `regf` signature check and validates the XOR checksum.
3. **Navigation Phase**:
   - The `SAMManager` starts at the `RootKeyOffset`.
   - It searches for "SAM" -> "Domains" -> "Account" -> "Users" -> "Names".
4. **Enumeration Phase**:
   - It lists every subkey in the `Names` folder.
   - It extracts the "Type" from each subkey (which Windows uses as the RID).
5. **Presentation Phase**:
   - The raw RIDs and names are formatted and inserted into the `ListView`.

4.2 Component-Based Architecture and Layering

The project is built on a modular design pattern to separate concerns.

4.2.1 The Core Components

1. **The Hive Module (`ntreg.h` / `ntreg.cpp`)**: This is the lowest layer. It doesn't know what a "User" is. It only knows how to follow offsets, find tags like `nk` or `vk`, and return raw data. It handles the binary "plumbing."
2. **The SAM Manager (`sam.h` / `sam.cpp`)**: This is the logic layer. It "knows" the meaning of the data. It knows where Windows stores user IDs and how to interpret the binary blobs.
3. **The UI Controller (`main.cpp`)**: This handles the user's clicks and keyboard input. It sends requests to the SAM Manager and displays the results.

5.3 Hash Lists and Optimization Strategies (lf, lh, ri, li)

As the Windows Registry grew in size and complexity, the speed of key resolution became a bottleneck. Moving from simple linear lists to sorted hash lists was a major architectural update.

5.3.1 The LF (Fast List) and LH (Hash List)

- **LF Records**: These store the first 4 characters of a subkey's name. When searching for a key named "System", the engine only checks keys whose first 4 bytes match "Syst". This drastically reduces the number of string comparisons.
- **LH Records**: These use a hash of the key's name instead of just the first 4 bytes. This provides even fewer collisions and faster lookups for keys with similar prefixes.

Our `ntreg.cpp` engine implements the specific logic required to parse these binary identifiers and jump to the correct child cell.

CHAPTER 6: TECHNICAL IMPLEMENTATION - SAM MANAGER

6.1 SAM Management (sam.cpp) Responsibilities

The `SAMManager` is the "brain" of the application when it comes to account security. While the `Hive` class sees the registry as a generic tree of folders and files, the `SAMManager` sees it as a security database. It must understand the "Meaning" of values that would otherwise look like random binary blobs.

6.2 Recursive Path Resolution and Tree Walking

Registry navigation is inherently recursive. To find the account data, the system must perform a coordinated "walk" through the tree.

1. **Resolve "SAM"**: Start at the root and find the SAM key.
2. **Resolve "Domains"**: Find the Domains subkey under SAM.
3. **Resolve "Account"**: Access the Account subkey.
4. **Resolve "Users"**: Target the main user container.
5. **Resolve "Names"**: List all account names.

This path resolution is implemented iteratively in our SAM module to prevent stack overflows on deeply nested, potentially malicious hives.

6.3 User Account Enumeration and RID Mapping

A common misconception is that the username is the primary identifier for a Windows account. In reality, Windows uses the **RID (Relative Identifier)**.

- **The Mapping Logic**: The `SAMManager` reads the username keys. Each key has a "Type" attribute (stored in the cell header) that contains the RID.
- **The Construction**: The RID is converted to an 8-digit uppercase hexadecimal string (e.g., 500 becomes `000001F4`).
- **The Final Jump**: The tool then looks for a key with that hex name in the `\Users` folder. This key contains the actual security records.

6.4 Forensic Analysis of V and F records

Windows stores the "real" user data in two values: **V** and **F**.

- **V (Variable)**: This is a complex, variable-length record. It starts with a 44-byte header that contains offsets to various fields (Username, Full Name, Comment, Home Directory, Hashes). Our tool parses this header to find the specific segments of the record.
- **F (Fixed)**: This is a 72-byte fixed-length structure. It contains:
  - **Timestamps**: Last login, last password change (8-byte FILETIME).
  - **Password Expiry**: When the user must next change their password.
  - **ACB (Account Control Bits)**: A bitmask that we manipulate to unlock accounts.

CHAPTER 8: MEMORY AND PERFORMANCE STRATEGIES

8.1 Buffer Safety and Bounds Checking

Working with binary files is dangerous. A corrupted or malicious hive could have offsets that point outside the file, leading to segmentation faults or "buffer underflow" exploits.

- **Safety Layer**: Our `getOffset` function performs a boundary check on every call. If an offset is larger than the total loaded size of the hive, the function returns `nullptr`, and the calling logic gracefully halts the operation.
- **Read-Only Integrity**: By default, the tool operates on a memory-copy of the hive. Changes are only committed back to disk if the user explicitly saves, protecting the original system data from accidental corruption.

8.2 Memory-Mapped vs. Buffered I/O

For the current version, we chose **Buffered I/O** (loading the entire hive into RAM).

- **Pros**: Blazing fast random access; simplified pointer arithmetic.
- **Cons**: High RAM usage for massive hives (e.g., 500MB+ SOFTWARE hives).
- **Future Improvement**: Implementing `CreateFileMapping` for "Lazy Loading" of data blocks.

CHAPTER 9: SOFTWARE TESTING AND QUALITY ASSURANCE

CHAPTER 7: WIN32 UI ARCHITECTURE

7.1 Dialog-Based Design Philosophy and UX Goals

Utility software should be functional and stay out of the user's way. For NTPWEdit Clone, we chose a **Dialog-Based Application** model rather than a standard window. Dialogs are easier to manage for fixed-layout tools and have built-in support for tab-navigation and default buttons.

The goal was to make the tool usable even by someone in a high-pressure recovery situation. The most common action (opening the system SAM) is automated, meaning the user only has to click one button to see the results.

7.2 The Win32 Message Pump and Native Controls

The application runs on the standard Windows message pump (`GetMessage` , `TranslateMessage` , `DispatchMessage`). This ensures that the application is natively responsive and uses the exact same UI engine as the operating system itself. By using native controls like `SysListView32` , we ensure that the tool respects the user's system theme and accessibility settings (like high-contrast mode).

CHAPTER 11: DISCUSSION AND SECURITY

11.1 Ethical Considerations in Security Software

Creating a tool that can list and potentially reset passwords brings up important ethical and legal considerations. Like a locksmith's set of picks, this tool can be used for both benevolent and malevolent purposes.

- **Benevolent Use**: A user regaining access to their family photos after forgetting a password.
- **Malevolent Use**: An unauthorized person gaining access to a private computer.

The development team's stance is that "security through obscurity" (relying on the file being hidden or locked) is not true security. By documenting these structures, we help administrators understand how to properly secure their systems.

11.2 The Impact of Full Disk Encryption (BitLocker)

It is important to note that this tool—and any offline registry parser—is rendered completely ineffective by **Full Disk Encryption** (like BitLocker, VeraCrypt, or FileVault). If the drive is encrypted, the binary SAM hive cannot even be found on the disk until the decryption key is provided. This reinforces the idea that FDE is a mandatory requirement for modern data privacy.

CHAPTER 12: CONCLUSION

The NTPWEdit Clone project has been a successful exploration into the internals of the Windows operating system. By building a custom registry parsing engine and a SAM management layer, we have created a functional utility that can solve real-world system recovery problems.

The technical journey—from parsing REGF headers to displaying user RIDs in a Win32 ListView—has provided deep insights into binary data management, systems programming in C++, and the security architecture of Windows NT.

GLOSSARY OF TERMS

- **ACB (Account Control Bits)**: A bitmask that controls account properties like "Disabled" or "Locked Out."
- **Binary Hive**: The physical file format of the Windows Registry.
- **Hole**: An unallocated cell within a registry bin.
- **Little-Endian**: The byte ordering used by modern x86 processors and the Windows Registry.
- **NTLM**: The standard network authentication protocol used by the SAM module.
- **RID (Relative Identifier)**: A unique number assigned to an account by the local SAM or Active Directory.

- **SysKey**: A master key used to protect password hashes inside the SAM hive.

REFERENCES

1. **Mark Russinovich**, "Windows Internals", 7th Edition, Microsoft Press.
2. **Microsoft Learn**, "Registry Hive Format", Official Documentation.
3. **RFC 1320**, "The MD4 Message-Digest Algorithm".
4. **Forensics Wiki**, "Registry Hive Format Forensic Specification".
5. **Petruska, J.**, "Internal Structure of the Windows Registry", 2003.